

Computer Networks — Sessional 1 Study Notes

Weeks 01–04: Introduction, Network Core, Security/Layering/History, Web & HTTP

Trimmed for exam efficiency, with fully solved quiz numericals

This version is trimmed for exam efficiency: the exam is conceptual + 2 numericals, so deep product/spec tables (Wi-Fi generations, cellular generations, fiber sub-types, frequency bands, careers, research topics) have been removed. Only what you can actually be asked – definitions, comparisons, and numerical formulas – remains.

Week 01 — Introduction to the Internet

1.1 What is the Internet?

Think of the Internet as a huge, worldwide web of roads (links) and intersections (routers/switches) that lets billions of devices talk to each other by sending small chunks of data back and forth.

Definitions worth memorizing (for short questions):

- **Kurose:** The Internet is a computer network that interconnects billions of computing devices throughout the world.
- **Forouzan:** An *internet* (lowercase i) is two or more networks that can communicate with each other. The *Internet* (uppercase I) is the most notable one, made of thousands of interconnected networks.
- **Tanenbaum:** A network of thousands of interconnected networks that use TCP/IP as the foundation for communication.
- **Peterson & Davie:** Internetworking connects diverse networks so any two hosts can communicate – this is the basis of the Internet.

Two views of the Internet:

1. **"Nuts and bolts" view:** The Internet = hosts (end systems) + packet switches (routers/switches) + communication links + protocols, all governed by Internet standards (RFCs, published by the IETF).
2. **"Services" view:** The Internet is infrastructure that provides services to applications (Web, email, video streaming) via a programming interface called **sockets**.

Socket analogy: A *socket* is like a "door" – a sending process pushes a message out its door, trusting the network (transport infrastructure) to carry it to the receiving process's door on the other side.

1.2 What is a Protocol?

A protocol is a **set of agreed rules** that two computers follow so they can understand each other – exactly like human conversation rules.

Human analogy: "Hi" → "Hi" → "Got the time?" → "2:00".

Formal definition (memorize this): Protocols define the **format** and **order** of messages sent and received among network entities, and the **actions** taken on message transmission and receipt.

An application-layer protocol defines:

- Types of messages exchanged (e.g. request, response)
- Message **syntax** – what fields exist and how they're separated
- Message **semantics** – meaning of the fields
- Rules for when/how processes send and respond to messages

Protocols can be **open** (defined in RFCs, e.g. HTTP, SMTP) or **proprietary** (e.g. Skype, Zoom).

1.3 Network Edge

The **network edge** = the end systems (hosts) that sit at the "edge" of the network.

- Hosts = run **application programs**.
- **Clients:** user-side devices (browsers, phones).
- **Servers:** powerful machines, often in big data centers.

1.4 Access Networks (concept only)

An **access network** connects an end system to its **first router** (the "edge router"). You just need to know the **categories** and be able to justify a choice – not memorize speed tables.

- **DSL:** uses existing telephone copper lines; asymmetric speed (download faster than upload); limited by distance to the telco office.
- **Cable (HFC):** uses cable-TV infrastructure; shared medium among neighbors.
- **FTTH (Fiber):** optical fiber to the home; fastest, most expensive; **AON** (active, dedicated fiber per user) vs **PON** (passive, shared fiber, cheaper/common).
- **5G Fixed Wireless:** wireless "last mile," no cabling needed, fast rollout.
- **Ethernet / Wi-Fi:** standard wired/wireless LAN access in homes and enterprises.

- **Cellular (3G/4G/5G):** wide-area wireless access via cell towers.

Exam angle: "Which access technology would you recommend for X scenario and why?" – always justify with *speed, cost, distance/coverage, and symmetry (up/down speed)*. You will NOT be asked exact Mbps figures for each standard.

1.5 Physical Media (concept only)

Guided media (signal travels through a solid medium): twisted-pair copper, coaxial cable, fiber optic (light pulses, lowest loss, immune to interference, most secure).

Unguided media (signal travels freely through air/space): terrestrial radio (mobile, but suffers path loss, shadowing, multipath fading, interference) and satellite (GEO = high delay, wide coverage; LEO = lower delay, needs many satellites).

1.6 Network Core

The **network core** = the mesh of interconnected **packet switches** (routers) and links that connects all the end systems together. Its job: **transport data** between the edges. The dominant technique is **packet switching**.

Switching: the big idea

Switching/forwarding = the technique used to move data from sender to receiver by choosing a path. Two main types:

	Circuit Switching	Packet Switching
Idea	Reserve a dedicated path (resources) for the entire "call".	Break the message into packets; each travels independently, sharing links on demand.
Resource sharing	None – resources are dedicated even if idle.	Shared, on-demand.
Setup	Requires call setup before data flows.	No call setup needed.
Performance	Guaranteed, circuit-like performance.	Variable delay; possible congestion, loss.
Multiplexing	FDM (Frequency Division) or TDM (Time Division).	Statistical multiplexing.
Used in	Traditional telephone networks.	The Internet.

FDM vs TDM:

- **FDM:** the frequency spectrum is divided into narrow bands; each call gets its own band and can transmit continuously within that band.
- **TDM:** time is divided into slots; each call gets a periodic time slot and can use the *full* frequency band, but only during its slot.

Packet Switching in detail

- Hosts break application messages into **packets**.
- Network forwards packets hop-by-hop (router to router) toward the destination.
- Each packet travels **independently**.
- **Store-and-forward:** a router must receive the *entire* packet before it can start forwarding it onward.

Transmission delay (core formula):

$$d_{trans} = \frac{L}{R}$$

where L = packet size in bits, R = link transmission rate in bits/sec. Time to **push all bits of a packet** onto the link.

Forwarding vs Routing

- **Forwarding** = *local* action: a router looks at the destination address in the arriving packet's header, checks its local forwarding table, and moves the packet to the correct output link.
- **Routing** = *global* action: routing algorithms determine the actual source-to-destination **paths** that packets take (they build the forwarding tables).

Memory trick: Routing = the "map-making" (global), Forwarding = "which door do I push this packet through right now" (local, per-router).

Queuing and Packet Loss

Routers have **output buffers (queues)**. If packets arrive faster than the link can transmit them, they wait – creating **queuing delay**. If the buffer is full when a new packet arrives, that packet is **dropped (lost)**.

Traffic intensity: $\frac{La}{R}$, where a = average packet arrival rate, L = packet length (bits), R = link bandwidth (bits/sec).

- $La/R \approx 0$: queueing delay is small.
- $La/R \rightarrow 1$: queueing delay grows large.
- $La/R > 1$: more work arriving than can be serviced – **delay becomes infinite**.

1.7 Numericals — Delay Formulas (classical, exam-relevant)

Example 1 (Packetization + Transmission Delay):

A host wants to send a message of size $M = 8$ Mb, broken into packets of $L = 1,000$ bits each, sent over a link with rate $R = 2$ Mbps.

$$\begin{aligned}\text{Number of packets} &= \frac{M}{L} = \frac{8,000,000}{1,000} = 8,000 \text{ packets} \\ \text{Delay per packet} &= \frac{L}{R} = \frac{1,000}{2,000,000} = 5 \times 10^{-4} \text{ s} = 0.5 \text{ ms} \\ \text{Total transmission delay (whole message)} &= \frac{M}{R} = \frac{8,000,000}{2,000,000} = 4 \text{ s}\end{aligned}$$

Example 2 (Store-and-forward, multiple links):

A packet of $L = 10,000$ bits travels source $\rightarrow$ router 1 $\rightarrow$ router 2 $\rightarrow$ destination (3 links), each rate $R = 100$ Mbps. Find total transmission delay (ignore propagation).

$$\begin{aligned}\text{Delay per hop} &= \frac{L}{R} = \frac{10,000}{100 \times 10^6} = 0.1 \text{ ms} \\ 3 \text{ links} \rightarrow \text{total delay} &\approx 3 \times 0.1 \text{ ms} = 0.3 \text{ ms}\end{aligned}$$

Example 3 (Transmission delay, given in bytes):

A packet of size 1,200 bytes is sent over a link with transmission rate 6 Mbps.

$$\begin{aligned}L &= 1200 \times 8 = 9,600 \text{ bits} \\ d_{trans} &= \frac{9,600}{6 \times 10^6} = 1.6 \times 10^{-3} \text{ s} = \mathbf{1.6 \text{ ms}}\end{aligned}$$

Example 4 (Propagation delay):

A link is 3,000 km long; propagation speed = 2.5×10^8 m/s.

$$d_{prop} = \frac{d}{s} = \frac{3,000,000}{2.5 \times 10^8} = 0.012 \text{ s} = \mathbf{12 \text{ ms}}$$

The master formula (memorize!):

$$d_{nodal} = d_{proc} + d_{queue} + d_{trans} + d_{prop}$$

d_{proc} : time to examine header, check errors, decide output link (given directly in word problems). d_{queue} : time waiting in the buffer (given directly, or use $D_q \approx \rho/(\mu(1-\rho))$). $d_{trans} = L/R$: time to push all bits onto the link. $d_{prop} = d/s$: time for a bit to physically travel the link.

Example 5 (Total nodal delay – combines everything):

A packet of 2,000 bytes experiences: processing delay = 2 ms, transmission rate = 4 Mbps, link length = 500 km, propagation speed = 2.5×10^8 m/s. Assume no queuing delay.

$$\begin{aligned}d_{proc} &= 2 \text{ ms} \\ L &= 2000 \times 8 = 16,000 \text{ bits}, \quad d_{trans} = \frac{16,000}{4 \times 10^6} = 4 \text{ ms} \\ d_{prop} &= \frac{500,000}{2.5 \times 10^8} = 2 \text{ ms} \\ d_{nodal} &= 2 + 0 + 4 + 2 = \mathbf{8 \text{ ms}}\end{aligned}$$

Example 6 (Queuing delay, M/M/1-style formula):

Average packet arrival rate $\lambda = 8$ packets/sec, service rate $\mu = 10$ packets/sec. Using $D_q \approx \frac{\rho}{\mu(1-\rho)}$ where $\rho = \lambda/\mu$:

$$\begin{aligned}\rho &= \frac{8}{10} = 0.8 \\ D_q &= \frac{0.8}{10 \times (1-0.8)} = \frac{0.8}{2} = \mathbf{0.4 \text{ s}}\end{aligned}$$

Example 7 (Packet switching vs circuit switching – capacity sharing):

A 1 Gbps link is shared. Each user needs 100 Mbps when "active," and is active only 10% of the time.

Circuit switching: Each user needs a dedicated 100 Mbps slice. Max users = $\frac{1000}{100} = \mathbf{10}$ users (fixed, regardless of activity).

Packet switching: Since each user is active only 10% of the time, many more than 10 users can share the link, because idle users don't waste capacity (statistical multiplexing).

Key takeaway: Packet switching is efficient for *bursty* data (resource sharing, no call setup) but can suffer *congestion* (delay, loss). Circuit switching gives guaranteed performance but wastes capacity when idle.

1.8 Throughput

Throughput = the rate (bits/time unit) at which bits are actually transferred between sender and receiver.

Bottleneck rule: end-to-end throughput is limited by the **slowest (bottleneck) link**:

$$\text{throughput} = \min(R_s, R_c, \dots)$$

For N connections sharing a backbone link of rate R fairly:

$$\text{per-connection throughput} = \min(R_c, R_s, R/N)$$

Example 8: 10 connections fairly share a backbone bottleneck link of $R = 100$ Mbps. Each connection's access links can do $R_s = 20$ Mbps and $R_c = 50$ Mbps.

$$\min(R_c, R_s, R/N) = \min(50, 20, 100/10) = \min(50, 20, 10) = \mathbf{10 \text{ Mbps}}$$

1.9 A Day in the Life of a Web Request (know the ORDER)

1. **DHCP:** laptop gets an IP address, first-hop router address, DNS server address.
2. **ARP:** laptop needs the router's **MAC address** to send it anything.
3. **DNS:** translate `www.google.com` into an IP address.
4. **TCP 3-way handshake:** $\text{SYN} \rightarrow \text{SYNACK} \rightarrow \text{ACK}$, connection established.
5. **HTTP request/response:** GET sent, page returned, displayed.

Order to remember: DHCP (get IP) $\rightarrow$ ARP (get router's MAC) $\rightarrow$ DNS (get server's IP) $\rightarrow$ TCP handshake (connect) $\rightarrow$ HTTP (get the actual page).

Week 02 — Internet Structure ("Network of Networks")

Step-by-step evolution of Internet structure (memorize this story!)

1. **Naive idea – connect every access ISP to every other directly.** **Problem:** doesn't scale – $O(N^2)$ connections for N ISPs.
2. **Better idea – connect each access ISP to one global transit ISP.** Customer ISPs pay the provider ISP (customer-provider relationship).
3. **Reality – competition.** Competing ISPs (A, B, C...) emerge; access nets connected only to ISP A can't easily reach ISP B's customers.
4. **Solution – peering & IXPs.** Competing ISPs connect directly via a **peering link**, often at an **IXP** (Internet eXchange Point).
5. **Regional ISPs emerge** to connect many smaller access networks up into the bigger ISPs economically.
6. **Content provider networks appear** – Google, Microsoft, Akamai build their own private networks to bring content closer to users, often bypassing regional/Tier-1 ISPs.

Final result – today's Internet structure: A small number of well-connected **Tier-1 ISPs** (e.g. Level 3, AT&T, NTT) at the "center," interconnected with each other. Around them: regional ISPs, access ISPs, IXPs, and content-provider private networks.

2.1 Applications and their Transport Protocols (Table to Memorize)

Application	Application-layer protocol	Transport protocol
File transfer/download	FTP	TCP
E-mail	SMTP	TCP
Web documents	HTTP	TCP
Internet telephony	SIP, RTP, or proprietary	TCP or UDP
Streaming audio/video	HTTP, DASH	TCP
Interactive games	Proprietary	UDP or TCP

Why TCP for most of these? File transfer, email, web pages need **100% reliable** delivery. Games/telephony often use UDP because low **delay** matters more than occasional lost packets.

3.1 Network Security

Key idea: The Internet was **not originally designed with security in mind**. The original vision was "a group of mutually trusting users attached to a transparent network." Security had to be added later, at every layer.

Common Attacks

Attack	What it is
Packet sniffing	On a broadcast medium (shared Ethernet, Wi-Fi), a device in promiscuous mode reads/records ALL packets passing by – including cleartext passwords.
IP spoofing	An attacker injects a packet with a false source address .
Denial of Service (DoS)	Attacker overwhelms a target with bogus traffic so legitimate users can't get service. Steps: select target → build a botnet → flood the target from compromised hosts (distributed = DDoS).

Lines of Defense

- **Authentication:** proving you are who you say you are.
- **Confidentiality:** via **encryption**.
- **Integrity checks:** digital signatures detect/prevent tampering.
- **Access restrictions:** password-protected VPNs.
- **Firewalls:** "middleboxes," typically **off-by-default**, filter incoming packets and help detect/react to DoS attacks.

3.2 Protocol Layers and Service Models

Why layering?

Networks are complex – layering organizes this complexity.

Airline travel analogy (know it well): Air travel = ticket purchase → baggage check → gate loading → takeoff → routing → ... → landing → gate unload → baggage claim. Each is a **layer** – it implements a service using its own actions, relying on the layer below.

Benefits of layering:

- Explicit structure allows identification of relationships between pieces.
- Modularization eases maintenance – a change inside one layer's *implementation* is transparent to the rest of the system.

The Internet Protocol Stack (5 layers – MUST memorize top to bottom)

Layer	Job	Example protocols
Application	Supports network applications	HTTP, IMAP, SMTP, DNS
Transport	Process-to-process data transfer	TCP, UDP
Network	Routing of datagrams from source to destination	IP, routing protocols
Link	Data transfer between neighboring network elements	Ethernet, 802.11 (Wi-Fi)
Physical	"Bits on the wire" – the raw signal	–

Memory trick (top to bottom): Application, Transport, Network, Link, Physical → "All Tigers Need Large Paws"

The OSI Model (7 layers) – know the 2 EXTRA layers

- **Presentation layer:** lets applications interpret meaning of data – encryption, compression, machine-specific data conventions.
- **Session layer:** synchronization, checkpointing, recovery of data exchange.

Important exam point: The Internet stack is "missing" these two layers – if needed, these services must be implemented *within the application layer* itself (e.g. TLS handles encryption at the application level).

Encapsulation – how data moves down and up the stack

Layer	Name of the data unit	What gets added
Application	Message (M)	The raw application data
Transport	Segment	M + transport header H_t
Network	Datagram	$[H_t M]$ + network header H_n
Link	Frame	$[H_n H_t M]$ + link header H_l

Analogy: Encapsulation is like **Matryoshka (Russian stacking) dolls** – each layer’s data sits nested inside the next layer’s wrapper. At the destination this runs in reverse (decapsulation) – each layer strips its own header and passes the rest up.

3.3 Internet History (KEY milestones only)

Period	Key events
1961–1972	Kleinrock (1961): queueing theory shows packet switching works. 1969: first ARPAnet node goes live. 1972: ARPAnet public demo; NCP = first host-host protocol; first email program.
1972–1980	Cerf and Kahn (1974) propose an architecture for interconnecting networks – principles (minimalism/autonomy, best-effort service, stateless routing, decentralized control) define today’s Internet architecture . 1976: Ethernet invented at Xerox PARC.
1980–1990	1983: TCP/IP deployed ; DNS defined. 1982: SMTP defined. 1985: FTP defined. 1988: TCP congestion control added.
1990, 2000s	Early 90s: the Web is born – HTML and HTTP invented by Tim Berners-Lee . 1994: Mosaic browser. Late 90s: Web commercialization; network security becomes a major concern.
2005–present	2008: Software-Defined Networking (SDN) introduced. Rise of high-speed wireless (4G/5G). Big providers (Google, Facebook) build private networks. Cloud computing (AWS, Azure).

If you remember only 5 facts: (1) Kleinrock’s queueing theory (1961) proved packet switching works. (2) ARPAnet’s first node went live in 1969. (3) Cerf and Kahn’s 1974 internetworking principles are the foundation of today’s Internet. (4) TCP/IP was deployed in 1983. (5) Tim Berners-Lee invented HTML/HTTP (the Web) in the early 1990s.

Week 04 — Web and HTTP

4.1 Web Basics

A **web page** is made of **objects** – each can live on a different web server. An object can be an HTML file, JPEG image, audio file, etc. A web page = one **base HTML file** + several **referenced objects**, each identified by a **URL (host name + path name)**.

4.2 HTTP Overview

HTTP is the Web’s application-layer protocol, following the **client-server model**: client (browser) requests/receives/displays objects; server sends objects in response.

HTTP uses TCP: client opens a TCP connection (socket) to the server (port 80) → server accepts → HTTP messages exchanged → connection closed.

HTTP is ”stateless” – the server keeps **no memory** of past client requests. Each request is independent. (Protocols that DO maintain state are much more complex – if server/client crashes, their views of ”state” can become inconsistent.)

4.3 Non-Persistent vs Persistent HTTP

	Non-Persistent HTTP	Persistent HTTP (HTTP/1.1)
Connections	TCP connection opened, at most 1 object sent, then closed. New connection per object.	TCP connection stays open; multiple objects sent over the SAME connection.

Response Time Formula (non-persistent HTTP, ONE object):

$$\text{Response time} = 2\text{RTT} + \text{file transmission time}$$

1 RTT to set up the TCP connection + 1 RTT for the HTTP request/first response bytes, plus the time to actually transmit the file.

Persistent HTTP: because the connection stays open, later objects need as little as **1 RTT total** for all referenced objects – roughly halving response time.

4.4 HTTP Methods – know all 4

Method	Purpose	Has Body?	Idempotent?	Typical use
GET	Retrieve data; parameters in URL; cacheable, bookmarkable; not secure for sensitive data	No	Yes	Fetching webpages, APIs
POST	Send data (form submissions, uploads); data in request body ; more secure; not cached	Yes	No	Logins, form submissions
HEAD	Same as GET but only headers , no body; faster	No	Yes	Checking metadata
PUT	Upload/replace a resource entirely	Yes	Yes	REST APIs – updating resources

”Idempotent” = doing the same request multiple times has the same effect as doing it once. GET, HEAD, PUT are idempotent; POST is **not**.

4.5 HTTP Status Codes – memorize these 5

Code	Meaning
200 OK	Request succeeded; requested object is in this response
301 Moved Permanently	Requested object has moved; new location in Location: header
400 Bad Request	Request message wasn't understood by the server
404 Not Found	Requested document not found on this server
505 HTTP Version Not Supported	Server doesn't support the HTTP version used

4.6 Cookies – Maintaining State

Since HTTP is stateless, **cookies** let websites/browsers remember information between visits.

Four components (classic short-question list):

1. Cookie header line in the HTTP **response** (**Set-Cookie:** ...)
2. Cookie header line in the next HTTP **request** (**Cookie:** ...)
3. A cookie file kept on the user's computer, managed by the browser
4. A back-end database at the website

First-party vs third-party cookies:

- **First-party:** set by the website you actually chose to visit.
- **Third-party:** set by a DIFFERENT site embedded inside the page you're visiting (e.g. an ad network) – lets it track you **across many websites**, which is how targeted ads work. Privacy concern: this tracking can be invisible to the user.

4.7 Web Caching (Proxy Servers)

A **web cache** sits between the browser and the origin server. Browser sends all requests to the cache.

- **Cache hit:** object IS in the cache → returned immediately (fast, only travels over the internal LAN).
- **Cache miss:** object NOT in cache → cache requests it from the origin server (full RTT + access-link transmission), stores a copy, returns it.

A web cache acts as **both a server** (to the client) **and a client** (to the origin server).

Why use web caching? Reduces response time (cache is closer), reduces traffic on the access link, helps smaller content providers deliver content effectively.

Web caching formulas (THIS IS A HIGH-PRIORITY NUMERICAL TOPIC – see solved quizzes below):

$$T_{\text{access}} = \frac{\text{Object Size}}{\text{Access Link Rate}}, \quad T_{\text{without cache}} = \text{RTT} + T_{\text{access}}$$

$$T_{\text{hit}} = \frac{\text{Object Size}}{\text{LAN Rate}}, \quad T_{\text{miss}} = \text{RTT} + T_{\text{access}}$$

$$T_{\text{with cache}} = (\text{Hit Rate} \times T_{\text{hit}}) + (\text{Miss Rate} \times T_{\text{miss}})$$

$$\text{Access-Link Utilization} = \frac{\text{Data Rate on Access Link}}{\text{Access Link Capacity}}, \quad \text{where Data Rate} = \text{Request Rate} \times \text{Object Size}$$

With a cache, only the *miss* requests travel over the access link, so: Data Rate (with cache) = (Request Rate × Miss Rate) × Object Size.

$$\% \text{ Reduction in Retrieval Time} = \frac{T_{\text{without}} - T_{\text{with}}}{T_{\text{without}}} \times 100$$

4.8 HTTP/2 and HTTP/3 (concept only)

Head-of-Line (HOL) blocking (HTTP/1.1 problem): server responds to GET requests **in order** (FCFS). If a client requests 1 large object and 3 smaller ones, the small objects must wait behind the large one.

HTTP/2 fix: objects are divided into **frames**, and frame transmission is **interleaved** – small objects get delivered quickly, with only slight delay to the large one, instead of waiting fully behind it. Server can also **push** objects proactively.

HTTP/3: HTTP/2 still runs over a single **TCP** connection, so one lost packet can stall ALL objects (TCP delivers in order). HTTP/3 runs over **UDP** instead, with its own per-object error/congestion control, avoiding this stall.

Solved Quiz Bank (Past Quizzes – Fully Worked)

These are your actual quiz questions, fully solved. Study the **pattern**: every delay-type numerical uses the same 4 formulas; every caching numerical uses the same 5 formulas from Section 4.7. If a numerical shows up in your sessional, it will look almost exactly like one of these.

Quiz 1(A) — Hospital Nodal Delay

Given: $L = 6,000$ bytes = 48,000 bits, $d_{proc} = 4$ ms, $d_{queue} = 6$ ms, $R = 12$ Mbps, $d = 2,000$ km, $s = 2.5 \times 10^8$ m/s.

A. Processing delay: given directly = 4 ms

B. Queuing delay: given directly = 6 ms

C. Transmission delay:

$$d_{trans} = \frac{L}{R} = \frac{48,000}{12 \times 10^6} = 0.004 \text{ s} = 4 \text{ ms}$$

D. Propagation delay:

$$d_{prop} = \frac{d}{s} = \frac{2,000,000}{2.5 \times 10^8} = 0.008 \text{ s} = 8 \text{ ms}$$

E. Total nodal delay:

$$d_{nodal} = d_{proc} + d_{queue} + d_{trans} + d_{prop} = 4 + 6 + 4 + 8 = \mathbf{22 \text{ ms}}$$

F. Largest component: Propagation delay (8 ms) contributes the most, because the link spans a long physical distance (2,000 km); queuing (6 ms) is second.

Component	Value
Processing Delay	4 ms
Queuing Delay	6 ms
Transmission Delay	4 ms
Propagation Delay	8 ms
Total Nodal Delay	22 ms

Quiz 1(B) — Bank Nodal Delay

Given: $L = 8,000$ bytes = 64,000 bits, $d_{proc} = 5$ ms, $d_{queue} = 8$ ms, $R = 16$ Mbps, $d = 2,500$ km, $s = 2.0 \times 10^{10}$ m/s (as given in the question).

A. Processing delay: given directly = 5 ms

B. Queuing delay: given directly = 8 ms

C. Transmission delay:

$$d_{trans} = \frac{L}{R} = \frac{64,000}{16 \times 10^6} = 0.004 \text{ s} = 4 \text{ ms}$$

D. Propagation delay:

$$d_{prop} = \frac{d}{s} = \frac{2,500,000}{2.0 \times 10^{10}} = 0.000125 \text{ s} = \mathbf{0.125 \text{ ms}}$$

E. Total nodal delay:

$$d_{nodal} = 5 + 8 + 4 + 0.125 = \mathbf{17.125 \text{ ms}}$$

F. Largest component: Queuing delay (8 ms) contributes the most here – because the propagation speed given is unusually high, propagation delay becomes almost negligible, so network congestion (queuing) dominates.

Component	Value
Processing Delay	5 ms
Queuing Delay	8 ms
Transmission Delay	4 ms
Propagation Delay	0.125 ms
Total Nodal Delay	17.125 ms

Pattern to notice (compare Quiz 1A and 1B): the dominant delay component changes depending on the numbers given – sometimes it's propagation (long distance, normal speed), sometimes it's queuing (heavy congestion). **Always compute all four and compare – don't assume.**

Quiz 2(A) — University Lab Web Cache

Given: Access link = 2 Mbps, LAN = 100 Mbps, RTT = 1.5 s, Object size = 120 Kbits = 120,000 bits, Request rate = 10 req/sec, Hit rate = 40%, Miss rate = 60%.

A. Retrieval time WITHOUT cache:

$$T_{\text{access}} = \frac{120,000}{2,000,000} = 0.06 \text{ s}$$

$$T_{\text{without}} = \text{RTT} + T_{\text{access}} = 1.5 + 0.06 = \mathbf{1.56 \text{ s}}$$

B. Retrieval time WITH cache:

$$T_{\text{hit}} = \frac{120,000}{100,000,000} = 0.0012 \text{ s} = 1.2 \text{ ms}$$

$$T_{\text{miss}} = \text{RTT} + T_{\text{access}} = 1.5 + 0.06 = 1.56 \text{ s}$$

$$T_{\text{with}} = (0.40 \times 0.0012) + (0.60 \times 1.56) = 0.00048 + 0.936 = \mathbf{0.93648 \text{ s}}$$

C. Access-link utilization WITHOUT cache:

$$\text{Data Rate} = 10 \times 120,000 = 1,200,000 \text{ bps} = 1.2 \text{ Mbps}$$

$$U_{\text{without}} = \frac{1.2}{2} = \mathbf{60\%}$$

D. Access-link utilization WITH cache: (only the 60% miss requests use the access link)

$$\text{Miss requests} = 10 \times 0.60 = 6 \text{ req/sec} \Rightarrow \text{Data Rate} = 6 \times 120,000 = 720,000 \text{ bps} = 0.72 \text{ Mbps}$$

$$U_{\text{with}} = \frac{0.72}{2} = \mathbf{36\%}$$

E. Percentage reduction in retrieval time:

$$\frac{1.56 - 0.93648}{1.56} \times 100 = \frac{0.62352}{1.56} \times 100 \approx \mathbf{39.97\% \approx 40\%}$$

F. Evaluation: The web cache reduces average retrieval time by almost 40%, and cuts access-link utilization from 60% down to 36%. This is a big improvement – lower utilization means shorter queues and less congestion risk at the access link, and the freed-up capacity can absorb more future traffic growth without needing an expensive link upgrade.

Quantity	Value
Retrieval Time Without Web Cache	1.56 s
Retrieval Time With Web Cache	0.93648 s (≈ 0.936 s)
Access-Link Utilization Without Cache	60%
Access-Link Utilization With Cache	36%
Percentage Reduction in Retrieval Time	$\approx 40\%$

Quiz 2(B) — Corporate Training Center Web Cache

Given: Access link = 5 Mbps, LAN = 100 Mbps, RTT = 1.2 s, Object size = 200 Kbits = 200,000 bits, Request rate = 12 req/sec, Hit rate = 50%, Miss rate = 50%.

A. Retrieval time WITHOUT cache:

$$T_{\text{access}} = \frac{200,000}{5,000,000} = 0.04 \text{ s}$$

$$T_{\text{without}} = \text{RTT} + T_{\text{access}} = 1.2 + 0.04 = \mathbf{1.24 \text{ s}}$$

B. Retrieval time WITH cache:

$$T_{\text{hit}} = \frac{200,000}{100,000,000} = 0.002 \text{ s} = 2 \text{ ms}$$

$$T_{\text{miss}} = \text{RTT} + T_{\text{access}} = 1.2 + 0.04 = 1.24 \text{ s}$$

$$T_{\text{with}} = (0.50 \times 0.002) + (0.50 \times 1.24) = 0.001 + 0.62 = \mathbf{0.621 \text{ s}}$$

C. Access-link utilization WITHOUT cache:

$$\text{Data Rate} = 12 \times 200,000 = 2,400,000 \text{ bps} = 2.4 \text{ Mbps}$$

$$U_{\text{without}} = \frac{2.4}{5} = \mathbf{48\%}$$

D. Access-link utilization WITH cache:

$$\text{Miss requests} = 12 \times 0.50 = 6 \text{ req/sec} \Rightarrow \text{Data Rate} = 6 \times 200,000 = 1,200,000 \text{ bps} = 1.2 \text{ Mbps}$$

$$U_{\text{with}} = \frac{1.2}{5} = \mathbf{24\%}$$

E. Percentage reduction in retrieval time:

$$\frac{1.24 - 0.621}{1.24} \times 100 = \frac{0.619}{1.24} \times 100 \approx \mathbf{49.9\% \approx 50\%}$$

F. Evaluation: The cache nearly halves both the average retrieval time and the access-link utilization (48% → 24%). This is an even stronger improvement than Quiz 2(A) because the hit rate here is higher (50% vs 40%), showing that **higher cache hit rates directly translate into bigger performance gains**.

Quantity	Value
Retrieval Time Without Web Cache	1.24 s
Retrieval Time With Web Cache	0.621 s
Access-Link Utilization Without Cache	48%
Access-Link Utilization With Cache	24%
Percentage Reduction in Retrieval Time	≈50%

5.1 Formula Sheet (everything you need for numericals)

$$\text{Transmission delay: } d_{trans} = \frac{L}{R} \quad (L = \text{bits}, R = \text{bits/sec})$$

$$\text{Propagation delay: } d_{prop} = \frac{d}{s} \quad (d = \text{link length}, s = \text{prop. speed})$$

$$\text{Total nodal delay: } d_{nodal} = d_{proc} + d_{queue} + d_{trans} + d_{prop}$$

$$\text{Traffic intensity: } \frac{La}{R} \quad (\text{if } > 1 \Rightarrow \text{delay} \rightarrow \infty)$$

$$\text{Queuing delay (M/M/1 approx.): } D_q \approx \frac{\rho}{\mu(1-\rho)}, \quad \rho = \frac{\lambda}{\mu}$$

$$\text{Bottleneck throughput: } \text{throughput} = \min(R_s, R_c, R/N)$$

$$\text{Non-persistent HTTP response time: } 2\text{RTT} + \text{file transmission time}$$

$$\text{Number of packets: } \frac{\text{Message size } M}{\text{Packet size } L}$$

$$\text{Web cache – no cache: } T_{\text{without}} = \text{RTT} + \frac{\text{Object Size}}{\text{Access Rate}}$$

$$\text{Web cache – with cache: } T_{\text{with}} = (\text{Hit} \times T_{\text{hit}}) + (\text{Miss} \times T_{\text{miss}})$$

$$\text{Access-link utilization: } U = \frac{\text{Request Rate} \times \text{Object Size}}{\text{Access Link Capacity}}$$

5.2 Definitions You Must Be Able to State Exactly

- Internet (Kurose's definition), protocol, forwarding vs routing, store-and-forward, traffic intensity, throughput, HTTP statelessness, idempotent, HOL blocking, cookie (4 components), IXP, Tier-1 ISP, OSI's 2 extra layers (presentation, session).

5.3 Classic Compare/Contrast Pairs

- Packet switching vs Circuit switching
- FDM vs TDM
- Forwarding vs Routing
- Non-persistent HTTP vs Persistent HTTP
- GET vs POST vs HEAD vs PUT
- First-party vs Third-party cookies
- HTTP/1.1 vs HTTP/2 vs HTTP/3
- Internet stack (5 layers) vs OSI model (7 layers)
- Cache hit vs Cache miss

5.4 The 2 Numericals – What to Expect

Based on the actual past quizzes, expect the **2 long numerical questions** to be variations of exactly these two question types:

1. **Nodal delay scenario** (like Quiz 1A/1B): given processing delay, queuing delay, packet size, link rate, distance and propagation speed as a story → compute all 4 delay components + total, then say which dominates.
2. **Web caching scenario** (like Quiz 2A/2B): given access-link rate, LAN rate, RTT, object size, request rate, and hit/miss percentages → compute retrieval time with/without cache, access-link utilization with/without cache, and % reduction.

Exam strategy tip: Always write down *which formula* you're using before plugging in numbers – partial credit is usually given for correct formula/setup even if the final arithmetic has a small error. Keep units consistent (bytes→bits, km→m, Kbits→bits, Mbps→bps) before calculating. For caching problems, remember: **WITH** a cache, only the **miss** fraction of requests ever touches the access link – this is the key insight for the utilization calculation.